

Technisches Architekturkonzept: Vollautomatisierte, KI-gesteuerte E-Learning-Plattform

Die Konzeption einer "Zero-Ops"-Plattform für E-Learning, welche hochgradig personalisierte Kurse aus Freitext-Eingaben generiert und diese als tägliche, videobasierte Lerneinheiten ausliefert, erfordert einen radikalen Paradigmenwechsel in der Softwarearchitektur. Wenn ein System autonom Wünsche in mehrwöchige, mediengestützte Lehrpläne übersetzen soll, stoßen traditionelle synchrone Architekturen, einfache Task-Queues und insbesondere visuelle No-Code-Automatisierungstools an ihre absoluten Grenzen. Die Orchestrierung von Large Language Models (LLMs), programmatischen Video-APIs und Vektordatenbanken erfordert eine strikte Code-First-Herangehensweise. Visuelle Tools wie n8n oder Make scheiden aufgrund mangelnder Versionierbarkeit in Git, fehlender Idempotenz-Garantien, limitierter Fehlerbehandlung bei verteilten Transaktionen und eingeschränkter Debugging-Fähigkeiten für komplexe "Long-Running Transactions" aus. Ein solches System muss auf Determinismus basieren, um die non-deterministischen Antworten von KI-Modellen verlässlich steuern zu können.

Die nachfolgende Architektur spezifiziert ein robustes, skalierbares und hochgradig automatisiertes Backend. Es ist speziell darauf ausgelegt, von modernen KI-Entwicklungsumgebungen (wie Cursor) und autonomen Agenten (wie Claude Code) verwaltet, skaliert und iteriert zu werden. Der inhaltliche Fokus liegt dabei auf der Vermittlung von IT-Grundlagen, KI-Schulungen und Cybersecurity, was wiederum höchste Ansprüche an den Datenschutz und die Manipulationssicherheit der Plattform stellt.

A. Backend-Architektur & Task-Orchestrierung

Die vollständige Automatisierung eines Kursgenerierungs-Prozesses stellt eine immense technische Herausforderung dar. Die Pipeline reicht von der initialen semantischen Skill-Analyse über die hierarchische Skripterstellung bis hin zum rechen- und zeitintensiven Rendering von Videos durch externe Dienste. Eine asynchrone Pipeline ist hierbei zwingend erforderlich, jedoch reichen traditionelle Message Broker allein nicht aus, um die erforderliche Ausfallsicherheit für langlebige Geschäftsprozesse zu garantieren.

Die systemischen Grenzen traditioneller Message Broker

Klassische Task-Runner und Queue-Systeme wie Celery (Python), Sidekiq (Ruby) oder BullMQ (Node.js) basieren in der Regel auf Redis oder RabbitMQ als Message Broker¹. Diese Broker sind hervorragend für kurze, zustandslose Hintergrundaufgaben (wie den Versand einer E-Mail oder die Skalierung eines Bildes) geeignet. Sobald ein Prozess jedoch aus Dutzenden von sequentiellen und parallelen API-Aufrufen besteht, die sich über Stunden erstrecken können, offenbaren sie gravierende strukturelle Schwächen. Wenn ein Task beispielsweise bei Schritt 47 von 50 abstürzt, beginnt ein nativer Task-Runner den gesamten Prozess von vorn, da ihm das

Konzept des feingranularen Zwischenspeicherns fehlt³.

In einer KI-gesteuerten E-Learning-Plattform führt dies unweigerlich zu doppelten API-Aufrufen, verschwendeten LLM-Tokens, überflüssigen Video-Renderings und letztendlich zu immensen finanziellen Kosten³. Zwar existieren PostgreSQL-native Queue-Systeme wie pgqueue oder graphile-worker, welche die Abhängigkeit von Redis eliminieren und transaktionales Enqueueing ermöglichen¹, jedoch lösen auch sie nicht das Problem der Workflow-Zustandsverwaltung bei langlebigen, verteilten KI-Agenten-Pipelines⁴. Bei komplexen Agenten-Workflows, die auf probabilistischen LLM-Antworten basieren, ist eine persistente Zustandsspeicherung des Ausführungspfades überlebenswichtig.

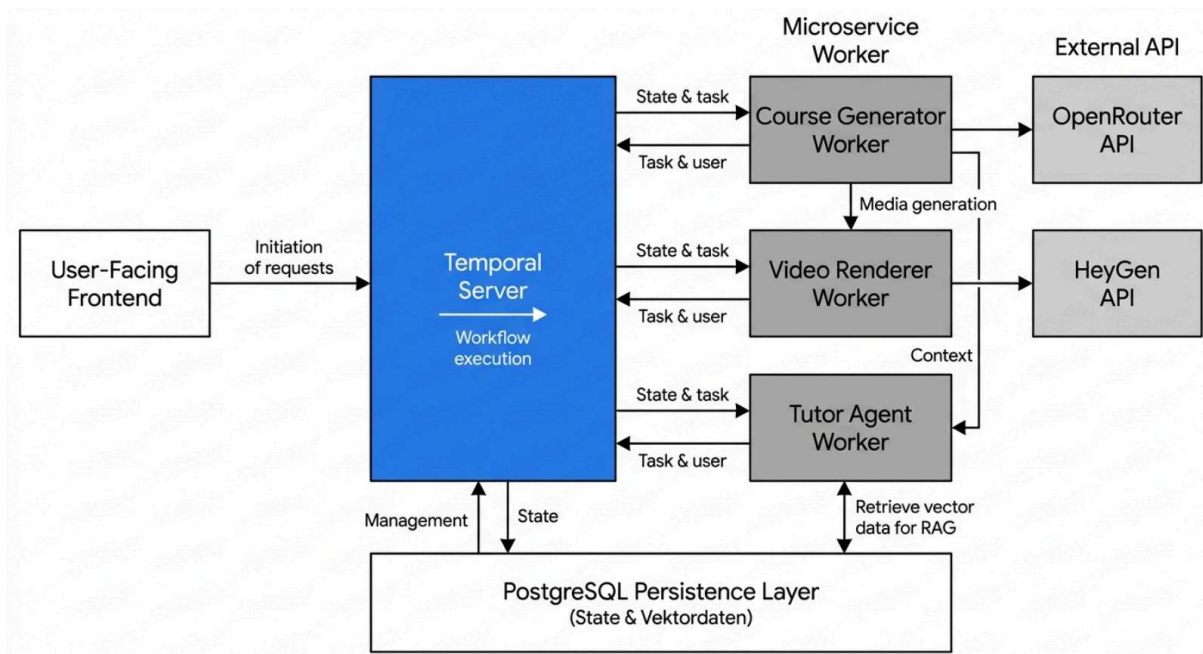
Durable Execution durch Temporal.io als zentrales Nervensystem

Um diese Herausforderungen zu meistern und echte "Zero-Ops"-Stabilität zu erreichen, wird die Integration von Temporal.io als zentraler Orchestrator zwingend vorgegeben⁶. Temporal abstrahiert die Komplexität verteilter Systeme vollständig und implementiert das Konzept der "Durable Execution" (dauerhafte Ausführung)⁷. Entwickler schreiben den Code so, als gäbe es keine Netzwerkausfälle, Serverabstürze oder API-Ratenlimits⁷.

Jeder Schritt in einem Temporal-Workflow wird automatisch in einer Datenbank (der Event History) protokolliert⁴. Stürzt der Worker-Prozess, der das Backend hostet, während der Generierung des achten von zehn Videomodulen aufgrund eines Out-of-Memory-Fehlers ab, wird der Workflow auf einem anderen Knoten im Cluster exakt dort fortgesetzt, wo er unterbrochen wurde³. Die Ergebnisse der ersten sieben Module werden schlicht aus der Event History von Temporal geladen, ohne die entsprechenden APIs (wie OpenRouter oder HeyGen) erneut anzusprechen³. Dies ist besonders bei teuren und ratenlimitierten LLM- und Video-APIs ein geschäftskritischer Vorteil.

Darüber hinaus separiert Temporal deterministischen Workflow-Code strikt von non-deterministischen Aktivitäten⁴. Ein LLM-Agent kann vollkommen dynamisch und kreativ agieren, Werkzeuge aufrufen und unvorhersehbare JSON-Strukturen generieren; Temporal sorgt lediglich dafür, dass diese Ausführungsschritte verlässlich persistiert werden und bei einem Systemfehler nicht verloren gehen⁴.

Architektur einer autonomen E-Learning-Plattform mit Durable Execution



Temporal.io orchestriert den gesamten Lebenszyklus der Kursgenerierung. Fällt ein Worker aus, übernimmt ein anderer nahtlos den Zustand aus dem Event-Log.

Fehlerkompensation durch das Saga-Pattern

Eine reine Vorwärts-Ausführung von Tasks reicht für komplexe Systeme nicht aus. Wenn eine mehrwöchige Kursgenerierung in der Mitte fehlschlägt (beispielsweise wegen extrem restriktiver Content-Richtlinien einer Video-API, die ein Skript zum Thema "Cybersecurity & Exploits" ablehnt), dürfen keine unvollständigen oder verwaisten Datenfragmente in der Datenbank verbleiben. Da relationale Datenbanken keine Transaktionen über Systemgrenzen hinweg (z. B. zwischen der eigenen Datenbank und der HeyGen-API) durchsetzen können, wird das Saga-Pattern implementiert¹⁰.

Eine Saga ist ein Architekturmuster für langlebige, verteilte Transaktionen, das bei Ausfällen automatisch Kompensationsaktionen (Compensating Actions) auslöst¹¹. Im TypeScript-SDK von Temporal wird dies in der Regel durch das Registrieren von Kompensationsfunktionen vor jedem kritischen Schritt realisiert¹⁰. Schlägt ein späterer Schritt im Workflow fehl, fängt eine übergreifende Fehlerbehandlungsroutine den Fehler auf und führt alle zuvor registrierten Kompensationen in umgekehrter Reihenfolge aus¹⁰. Dies könnte bedeuten, dass bereits angelegte Datenbankeinträge für Kursmodule wieder gelöscht und initiierte API-Calls zur Videogenerierung storniert werden¹⁰. Wichtig ist hierbei die Idempotenz der

Kompensationsfunktionen: Sie müssen so geschrieben sein, dass sie beliebig oft ausgeführt werden können, ohne Nebeneffekte zu erzeugen, selbst wenn die ursprüngliche Aktion nie vollständig abgeschlossen wurde¹⁰. Dies garantiert die Systemkonsistenz und führt das System auf den "Last Known Good State" zurück, ohne dass ein Administrator manuell eingreifen muss¹¹.

Best Practices für die modulare Erweiterbarkeit

Um die Plattform später modular erweitern zu können (z. B. durch Hinzufügen weiterer Video-Provider, zusätzlicher LLM-Modelle oder neuer Evaluierungsagenten), müssen spezifische Best Practices der verteilten Architektur befolgt werden. Die Isolation der Worker anhand ihrer fachlichen Domänen ist unerlässlich. Ingestion-Worker (die Freitext interpretieren), Enrichment-Worker (die Skripte und Videos generieren) und Sync-Worker (die Datenbanken aktualisieren) sollten auf separaten Temporal Task Queues laufen⁸. Dies verhindert, dass ein ressourcenintensiver Vorgang (wie das Herunterladen großer Videodateien) die Queue für schnelle LLM-Agenten-Routinen blockiert und diese verhungern lässt⁸.

Ein weiteres kritisches Entwurfsmuster betrifft den Datenaustausch zwischen den Workflow-Schritten. Temporal Payloads (die Daten, die in der Event History gespeichert werden) haben strikte Größenbeschränkungen⁸. Für große Datenmengen, wie etwa komplette Kurs-Metadaten im JSON-Format oder gar heruntergeladene Videoblob-Fragmente, darf Temporal nicht als Datenspeicher missbraucht werden. Stattdessen agiert Cloud-Storage (wie AWS S3 oder ein selbstgehosteter MinIO-Cluster) als Datenbus: Die Aktivität schreibt die Datei in den Object Storage und übergibt lediglich die URI an den Temporal-Workflow, welcher diese an die nächste Aktivität weiterreicht⁸. Diese Entkopplung hält den Workflow leichtgewichtig, performant und extrem gut testbar.

B. KI-Stack & Content-APIs

Die Generierung von inhaltlich hochwertigen, kohärenten und interaktiven Kursen erfordert ein orchestriertes Zusammenspiel aus extrem performanten, globalen LLMs für komplexe Strukturierungsaufgaben und datenschutzfreundlichen lokalen LLMs für sensible Analysen.

Strukturierte Lehrplangenerierung via Instructor und OpenRouter

Um aus einem formlosen Freitext-Wunsch eines Schülers (z. B. "Ich möchte in drei Wochen die Grundlagen von Kubernetes und Docker lernen, habe aber nur wenig Vorwissen in Linux") einen detaillierten, maschinenlesbaren Lehrplan zu erstellen, muss das Backend deterministische JSON-Strukturen aus naturgemäß non-deterministischen Sprachmodellen erzwingen. Die Nutzung der instructor-Bibliothek in Kombination mit Pydantic-Modellen ist hierfür der De-facto-Standard in Python-basierten (oder analogen TypeScript/Zod-basierten) KI-Architekturen¹⁵.

Über einen API-Multiplexer wie OpenRouter wird auf führende Frontier-Modelle wie Claude 3.5 Sonnet (hervorragend im Programmieren und Strukturieren) oder GPT-4o zugegriffen¹⁷. Die Herausforderung besteht darin, dass Modelle selbst bei strikten Prompts gelegentlich

JSON-Strukturen verletzen oder erforderliche Schlüssel weglassen. Die instructor-Bibliothek löst dies, indem sie die zugrundeliegenden APIs (etwa die OpenAI-kompatiblen Endpunkte von OpenRouter) modifiziert und das Modell über Tools (Function Calling) oder spezielle JSON-Modi strikt an ein vordefiniertes Pydantic-Schema bindet¹⁶.

So wird garantiert, dass die Antwort immer ein tief verschachteltes, validierbares Array von Lektionen enthält. Das Pydantic-Modell definiert dabei exakt die benötigten Metadaten, die Teleprompter-Skripte für den Video-Avatar, die Multiple-Choice-Fragen und die dazugehörigen Musterlösungen. Dies eliminiert fehlerhafte JSON-Syntax beinahe vollständig und reduziert den Parsing-Overhead im Backend auf ein absolutes Minimum¹⁵. Sollte das Modell dennoch halluzinieren und einen fehlerhaften Typ zurückgeben (etwa einen String statt eines Integers für die Kursdauer), fängt Pydantic den Validierungsfehler ab, und instructor führt automatisch einen Retry-Aufruf an das LLM durch, bei dem der Fehlermeldungstext als Korrekturhinweis mitgegeben wird¹⁵.

DSGVO-konforme Inferenz durch lokale LLMs (vLLM)

Während generische Aufgaben zur Strukturierung von Kursen problemlos und kosteneffizient über globale Provider via OpenRouter abgewickelt werden können, erfordern datenschutzsensible Interaktionen maximale Datensouveränität. Wenn Schüler im Rahmen eines Cybersecurity-Kurses firmeninternen Code zur Prüfung einreichen, Skill-Analysen basierend auf Schwächen durchgeführt werden oder proprietäre Unternehmensrichtlinien im RAG-Kontext diskutiert werden, verbietet sich der Transfer dieser Daten an US-Cloud-Konzerne oftmals strikt aufgrund von DSGVO, ISO 27001 oder internen Compliance-Vorgaben¹⁹.

Für diese sensiblen Tutor-Aufgaben wird zwingend ein Self-Hosted-Ansatz auf Servern innerhalb der EU (bevorzugt Deutschland) gewählt¹⁹. Die Ausführung von Modellen wie Mistral, Llama 3 (z. B. in der 8B- oder 70B-Variante) oder DeepSeek erfordert eine dedizierte Inference-Infrastruktur. Während Frameworks wie Ollama oder LM Studio sich hervorragend für das Prototyping oder kleine Solo-Deployments eignen²⁰, brechen sie unter Produktionslast mit vielen parallelen Anfragen oft zusammen.

Für den produktiven Hochlastbetrieb wird daher vLLM via Docker auf dedizierten NVIDIA-GPU-Clustern vorgegeben²¹. vLLM ist eine hochoptimierte Inference-Engine, die speziell auf extremen Durchsatz ausgelegt ist²².

Feature	Traditionelle LLM-Runtimes (HuggingFace, Standard Ollama)	vLLM (Production Backend)
Speicherverwaltung	Statische Allokation führt zu massiver Fragmentierung.	Nutzt PagedAttention zur dynamischen Zuweisung des Key-Value (KV) Caches,

		eliminiert Fragmentierung nahezu vollständig. ²⁴
Batching	Statisches Batching (wartet, bis alle Sequenzen eines Batches beendet sind).	Continuous Batching erlaubt das dynamische Hinzufügen neuer Requests in laufende Inferenz-Schleifen. ²⁵
Durchsatz	Gering, hohe Latenz bei vielen Nutzern.	14-24x höherer Durchsatz bei gleichzeitig stark optimierter Speicherauslastung. ²⁴
Quantisierung	Limitiert, oft zulasten der Latenz.	Native Unterstützung für FP8, AWQ, und GPTQ für maximale Geschwindigkeit auf H100/A100 GPUs. ²⁴
API-Standard	Oft proprietär.	Vollständige OpenAI-Kompatibilität, nahtloser Austausch im Code. ²⁴

Durch den Einsatz von vLLM können hunderte parallele Chat-Sitzungen der Schüler vom KI-Tutor bedient werden²⁵. Die Bereitstellung erfolgt als zustandsloser Docker-Container, idealerweise orchestriert über Kubernetes (sofern das Setup skaliert werden muss), wobei der Speicher über den Host-IPC-Namespaces geteilt wird (--ipc=host), um Inter-Prozess-Kommunikationsfehler bei großen Modellen zu vermeiden²⁴.

Programmatische Video-Erstellung über Webhook-Workflows

Die tägliche "Drip"-Auspielung der Inhalte – ein bewährtes didaktisches Konzept zur Vermeidung von kognitiver Überlastung – erfordert die vollautomatisierte Erstellung von Schulungsvideos aus den generierten Skripten. Anbieter wie HeyGen oder Synthesia bieten hierfür leistungsstarke REST-APIs an, mit denen lebenssechte KI-Avatare programmatisch gesteuert werden können³⁰. Da das Rendering eines hochauflösenden einminütigen Videos serverseitig in der Regel ein bis zwei Minuten beansprucht, ist ein asynchroner Ablauf bei der Generierung unabdingbar³⁰.

Ein nativer Code-First-Ansatz verzichtet auf ineffizientes Polling (das minütliche Abfragen des Status über GET /v3/videos/{video_id}) und nutzt stattdessen ereignisgesteuerte Webhook-Architekturen³⁰. Der Workflow stellt sich wie folgt dar: Das Backend nutzt den Temporal Orchestrator, um eine POST-Anfrage an die HeyGen Video-Agent API zu senden. Dabei werden das Skript, die ausgewählte avatar_id, die voice_id und ein eindeutiger

callback_id Parameter übergeben, der das Video logisch mit der Lektion in der PostgreSQL-Datenbank verknüpft³¹. Sobald die Anfrage abgesetzt ist, pausiert der Temporal-Worker diese spezifische Aktivität ressourcenschonend. Sobald die Server des Anbieters das Video fertiggestellt haben, sendet HeyGen einen asynchronen POST-Request an den registrierten Webhook-Endpunkt des E-Learning-Backends, beispielsweise mit dem Event-Typ avatar_video.success³⁴. Ein extrem kritischer Faktor in diesem Workflow ist die Sicherheit: Der Payload des Webhooks muss kryptographisch verifiziert werden, um zu verhindern, dass unautorisierte Akteure Endpunkte aufrufen und Video-Abschlüsse fingieren oder Schadcode einschleusen³⁴. HeyGen signiert den rohen Request-Body mit einem HMAC-SHA256-Hash unter Verwendung eines initial generierten Endpoint-Secrets³⁴. Das Backend liest diesen Payload, berechnet den Hash-Wert selbstständig nach und vergleicht das Ergebnis (in zeitlich konstanter Ausführung mittels kryptographischer Bibliotheken, um Timing-Angriffe zu verhindern) mit der im Header mitgelieferten Signatur X-Signature³⁴. Erst nach erfolgreicher Validierung weckt das Backend den wartenden Temporal-Workflow über ein Signal auf, extrahiert die video_url aus dem Payload (welche oft als zeitlich limitierter Pre-Signed-Download-Link fungiert) und persistiert das finale Videodokument in einem eigenen Cloud-Storage-Bucket (wie AWS S3), um Abhängigkeiten von externen CDN-Laufzeiten zu eliminieren³⁰.

C. Tutor-System & RAG-Implementierung

Ein KI-Tutor (wie ein Chat-Agent, der Übungen korrigiert und Fragen beantwortet) ist in einem E-Learning-Kontext nur so nützlich wie das spezifische Fachwissen, das er heranziehen kann. Allgemeine Sprachmodelle neigen gerade bei hochspezifischen IT-Themen, proprietären Plattforminhalten oder neu generierten Kursen zur Halluzination. Die Implementierung eines präzisen Retrieval-Augmented Generation (RAG) Systems löst dieses Problem, indem der Agent stets gezwungen wird, exakt auf Basis der zugrundeliegenden, generierten Kursinhalte zu antworten.

Vektordatenbank-Evaluation: pgvector vs. dedizierte Systeme

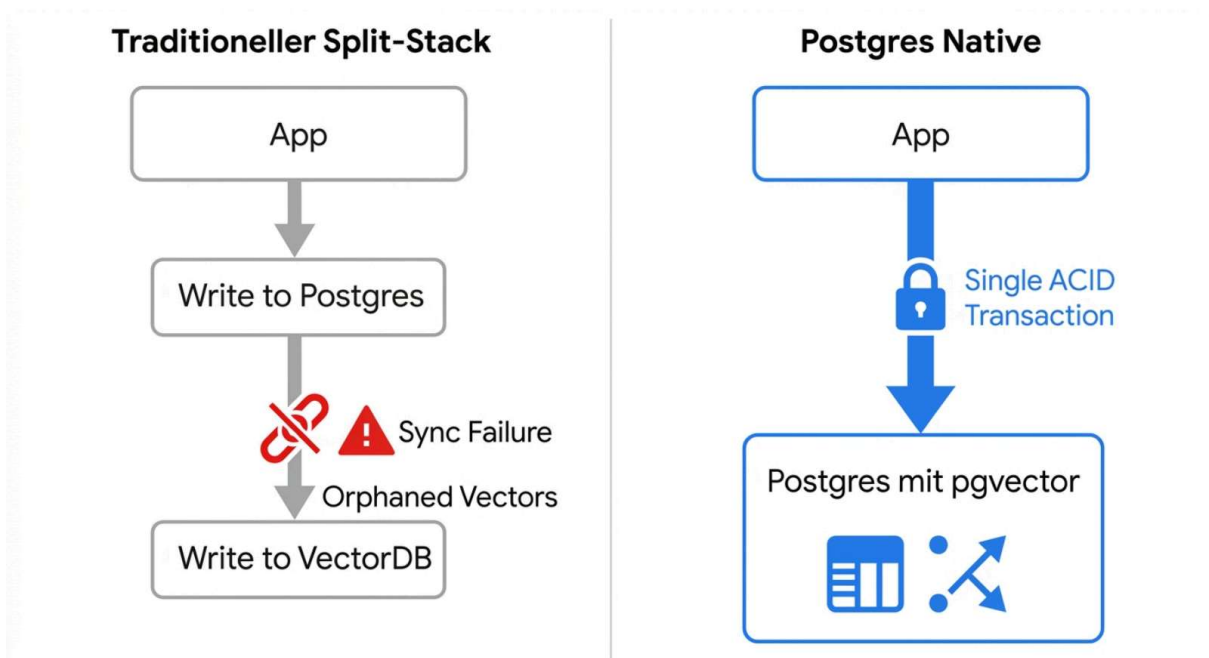
Für die Speicherung und semantische Durchsuchbarkeit von Vektoreinbettungen (Embeddings) der textuellen Kursinhalte stehen Architekten typischerweise vor der Wahl zwischen dedizierten Vektordatenbanken (wie Qdrant, Pinecone, Weaviate) oder Erweiterungen bestehender relationaler Datenbanken (wie pgvector in PostgreSQL)³⁸.

Für diese geforderte "Zero-Ops"-Architektur wird pgvector uneingeschränkt priorisiert. Der Verzicht auf eine separate, dedizierte Vektordatenbank reduziert die Systemkomplexität und den betrieblichen Wartungsaufwand erheblich⁴⁰. In einem System, in dem Tausende von asynchronen Kursgenerierungs-Jobs parallel laufen, müssen Transaktionen zwingend konsistent bleiben. Nutzt man eine externe Vektordatenbank, entsteht ein massives Risiko asynchroner Zustände (Split-System Architecture): Wenn der Eintrag der Lektions-Metadaten im relationalen Kurs-Repository erfolgreich war, der Upload der zugehörigen Vektoren in die externe Datenbank jedoch aufgrund eines Netzwerk-Timeouts fehlschlug, existieren verwaiste

Relationen⁴¹. Ein solches System erfordert komplexe, fehleranfällige Synchronisations-Skripte und externe Background-Jobs zur Bereinigung⁴¹.

Mit pgvector leben die relationalen Kursmetadaten, die eigentlichen Text-Chunks und ihre hochdimensionalen Vektoren in derselben Tabelle und können innerhalb einer einzigen ACID-Transaktion manipuliert werden⁴¹. Dies reduziert die Latenz für metadatengefilterte Abfragen massiv und eliminiert das sogenannte "Inconsistency Window" zwischen Metadaten-Updates und Vektor-Updates vollständig⁴².

Vektor-Persistenz: Eliminierung von Sync-Fehlern durch pgvector



Durch die Konsolidierung von Metadaten und Vektoreinbettungen in PostgreSQL (pgvector) werden ACID-Garantien gewahrt. Das Risiko inkonsistenter Datenbankzustände über Systemgrenzen hinweg entfällt.

Mandantenfähigkeit und Vermeidung von Halluzinationen

Ein kritisches Sicherheits- und Reputationsrisiko in dynamischen E-Learning-Plattformen ist das sogenannte "Retrieval Leakage" – die Situation, in der das RAG-System des KI-Tutors bei einem Studenten versehentlich Wissen aus dem privaten Kursmaterial oder gar den hochgeladenen Übungs-Codes eines anderen Studenten heranzieht⁴⁰. Solche Datenlecks zerstören das Vertrauen von Unternehmenskunden, die ihre Mitarbeiter schulen lassen, augenblicklich. Die Architektur löst dieses Problem durch strikte Mandantenisolierung (Multi-Tenant Isolation) direkt in den Index-Strukturen der Vektordatenbank⁴⁰. Anstatt die Filterung erst in der

Applikationslogik nach dem Abruf aller semantisch ähnlichen Vektoren durchzuführen (sogenanntes Post-Filtering, was extrem ineffizient ist und oft zu ungenauen Ergebnissen führt, da relevante Chunks verworfen werden), nutzt das System native Metadaten-Filterung³⁸. In pgvector wird dies durch einen kombinierten Index erreicht. Ein HNSW-Index (Hierarchical Navigable Small World), der für schnelle approximative Nächste-Nachbarn-Suchen optimiert ist, wird gepaart mit einer Row-Level Security (RLS) Policy oder direkten Filterbedingungen auf einer tenant_id und course_id Spalte⁴¹. Dadurch wird der mathematische Suchgraph der Vektor-Suche bereits im Vorfeld auf die Daten des spezifischen Nutzers beschnitten. Als Embedding-Modelle kommen hierbei leistungsstarke Modelle wie text-embedding-3-small (mit 1536 Dimensionen) von OpenAI zum Einsatz, oder für strikte DSGVO-Setups lokale Modelle wie bge-m3 oder nomic-embed-text-v1.5, die über Ollama gehostet und für den Abgleich genutzt werden können⁴⁴. Zusätzlich unterstützt Postgres-native Erweiterungen wie pg_trgm hybride Suchen, die semantische Vektorähnlichkeit mit exakter Volltextsuche (Keyword-Matching) kombinieren⁴⁴. Der finale RAG-Prompt an das LLM erhält somit ausschließlich hochrelevante, isolierte Kontextfragmente. Das lokale vLLM referenziert diese Chunks, bewertet die spezifische Code-Eingabe des Nutzers präzise und antwortet kontextsensitiv, ohne externe Datenstrukturen zu halluzinieren.

D. Manipulationssicheres Time-Tracking

Die exakte und auditsichere Erfassung der Lernzeit ist für zertifizierte Weiterbildungen, staatlich geförderte Maßnahmen und den Nachweis von Compliance-Trainings (insbesondere im regulierten Umfeld der Cybersecurity) essenziell. Die Erfassung der reinen Sitzungsdauer – also lediglich der Zeitstempel vom Login bis zum Logout – ist eine unzureichende und leicht fälschbare Metrik, da sie passive "AFK"-Zeiten (Away From Keyboard) oder das einfache Offenlassen eines Tabs im Hintergrund nicht herausfiltert.

Client-seitige Erfassung und Server-Validierung

Die robuste Erfassung beginnt auf der Client-Seite durch die Implementierung eines intelligenten "Heartbeat"-Mechanismus. Das React- oder Vue-Frontend sendet nicht kontinuierlich Datenpakete, was den Server überlasten würde, sondern aggregiert Aktivitätsereignisse lokal im Browser⁴⁵. Durch das Abhören von DOM-Events (wie Scrollen, Mausbewegungen und Tastaturanschläge) verifiziert das Frontend, ob der Nutzer tatsächlich physisch anwesend ist. Parallel dazu wird die Sichtbarkeit des Browser-Tabs über die moderne Page Visibility API (document.visibilityState) streng überwacht. Wechselt der Student den Tab, um auf anderen Webseiten zu surfen, minimiert das Fenster oder ist für mehr als eine definierte Toleranzzeit (z. B. 60 Sekunden) vollständig inaktiv, wird der Heartbeat-Timer auf dem Client rigoros pausiert⁴⁵.

In definierten Intervallen (beispielsweise alle 3 bis 5 Minuten) berechnet das Frontend die kumulierte, tatsächliche Aktivitätszeit und sendet einen Heartbeat-Payload an das Backend⁴⁵. Der Server vertraut diesen Daten jedoch nicht blind, sondern validiert die Anfragen. Ein Server-seitiger "Speed Limit"-Check verhindert, dass ein kompromittiertes oder manipuliertes Frontend künstliche Heartbeats in Hochgeschwindigkeit sendet, um Lernzeit in unnatürlich

kurzen Abständen massiv in die Höhe zu treiben.

Kryptographische Hash-Ketten für Revisionssicherheit

Um zu verhindern, dass kompromittierte Datenbank-Administratoren oder Angreifer die Tabelle der erfassten Zeiten im Nachhinein manipulieren (z. B. um Mindestlernzeiten für den Erhalt eines Zertifikats künstlich zu erreichen), wird ein manipulationssicheres Log-System mittels SHA-256 Hash-Verkettung (Hash Chaining) direkt auf Datenbankebene implementiert⁴⁶.

Dieses Prinzip, ähnlich der Struktur einer Blockchain, verknüpft jeden Heartbeat-Eintrag in der Datenbank kryptographisch mit dem vorherigen Eintrag⁴⁶. Der Algorithmus berechnet den aktuellen Hash aus einer Konkatenation der Felder SessionID, Timestamp, ActiveDuration, und PreviousHash⁴⁶. Verändert ein Angreifer nun nachträglich auch nur einen einzigen Zeitstempel in der Historie, stimmt der Hash dieses Eintrags nicht mehr, wodurch die gesamte nachfolgende Kette unwiderruflich invalidiert wird. Eine regelmäßige Server-seitige Überprüfungsroutine validiert die Integrität dieser Kette im Hintergrund und meldet Diskrepanzen sofort an das Admin-Dashboard. Experimentelle Evaluierungen solcher Architekturen zeigen, dass sie bei einer sehr geringen Systemlatenz eine nahezu 99-prozentige Genauigkeit bei der Betrugserkennung und Manipulationsverhinderung erreichen und somit strengste Audit- und Compliance-Anforderungen erfüllen⁴⁶.

Stateless Security und strikte JWT-Validierung

Für die Autorisierung dieser kritischen Heartbeats und sämtlicher anderer API-Aufrufe der Plattform kommen JSON Web Tokens (JWT) zum Einsatz. JWTs sind signierte Tokens, die Identitäts- und Berechtigungsinformationen enthalten und eine zustandslose (stateless) Skalierung des Backends ermöglichen, da die Validierung erfolgen kann, ohne bei jedem Request die Session-Datenbank abzufragen⁴⁷. Sie bergen jedoch erhebliche Sicherheitsrisiken, wenn sie vom Backend nicht extrem strikt validiert werden⁴⁸.

Das Architekturkonzept sieht vor, dass das Backend sich nicht auf oberflächliche Client-Dekodierungen oder bloße Ablaufprüfungen verlässt. Bei jedem eintreffenden Token durchläuft das System eine mehrstufige Validierungs-Pipeline⁴⁹:

1. **Signatur-Verifikation:** Prüfung des Tokens gegen einen serverseitig sicher aufbewahrten und rotierenden Public Key (oft abgerufen über einen JWKS – JSON Web Key Set Endpunkt), um Manipulationen am Payload absolut auszuschließen (Tamper-Proof)⁴⁷.
2. **Ablaufdatum (exp):** Striktes Ablehnen von abgelaufenen Tokens, um das Zeitfenster für Replay-Angriffe bei gestohlenen Sitzungen zu minimieren. Ein JWT-Access-Token sollte idealerweise eine sehr kurze Lebensdauer (z.B. 15 Minuten) haben und über Refresh-Tokens aktualisiert werden⁴⁷.
3. **Audience (aud) und Issuer (iss):** Sicherstellung, dass das Token explizit vom korrekten Autorisierungsserver (iss) und exakt für diesen spezifischen E-Learning-Microservice (aud) ausgestellt wurde, um Cross-Service-Angriffe zu unterbinden⁴⁹.
4. **Token-Typ-Differenzierung:** Strikte Unterscheidung zwischen ID-Tokens (zur Nutzerauthentifizierung) und Access-Tokens (zur API-Autorisierung), da die Vermischung

dieser Kontexte zu schwerwiegenden Sicherheitslücken führen kann⁴⁹.
Durch diese Kombination aus robuster kryptographischer JWT-Authentifizierung am Rand des Netzwerks, verifizierten Frontend-Metriken und der Hash-Chaining-Sicherung tief in der Postgres-Datenbank entsteht ein lückenlos geschlossenes System, das absolut verlässliche und auditsichere Lernzeitmetriken garantiert.

E. Technischer Blueprint und Datenmodelle

Um das Architekturkonzept abzuschließen und die Brücke zur praktischen Entwicklung zu schlagen, werden im Folgenden die relationalen Datenmodelle und der genaue Ablaufplan eines Kursgenerierungs-Jobs definiert. Dies dient als direkter technischer Blueprint für die Implementierung mit modernen ORM (Object-Relational Mappers wie Prisma oder Drizzle) und KI-Entwicklungsumgebungen wie Cursor.

Relationales Datenbankschema (PostgreSQL)

Die Datenstrukturen sind konsequent für eine hochperformante Abfrage, strikte Mandantenisolierung und effiziente RAG-Nutzung optimiert.

Entität	Felder & Typen	Beschreibung / Funktion
Users	id (UUID, PK), role (Enum), email (String, Unique), tenant_id (UUID)	Kernentität für die Zugriffskontrolle. Rollen umfassen student, admin, system.
Courses	id (UUID, PK), user_id (FK), status (Enum), topic (String)	Status kann generating, pending_approval, active oder failed sein.
Modules	id (UUID, PK), course_id (FK), sequence_order (Int), title (String)	Logische Strukturierungseinheit des Kurses für mehrwöchige Lehrpläne.
Lessons	id (UUID, PK), module_id (FK), content_payload (JSONB), video_url (String)	Enthält das von Pydantic/Instructor generierte und validierte JSON für Text und Quizfragen.
Embeddings	id (UUID, PK), lesson_id (FK), tenant_id (FK), embedding	Die pgvector-Tabelle für das RAG-System. Nutzt einen HNSW-Index über die

	(Vector: 1536)	Vektorspalte.
ActivityLogs	id (UUID, PK), session_id (String), duration_sec (Int), crypto_hash (String)	Manipulationssicheres Log der Heartbeats. crypto_hash = SHA256 Kette für Audits.

Workflow-Spezifikation des Kurs-Generators (Temporal Saga)

Der folgende Ablauf veranschaulicht eine beispielhafte Ausführung innerhalb des Temporal.io TypeScript SDKs. Er zeigt auf, wie der asynchrone Prozess robust orchestriert wird, um den "Zero-Ops"-Anspruch des Betreibers zu gewährleisten und manuelle Eingriffe überflüssig zu machen.

1. **Initiierung (Workflow Start):** Der Schüler (oder das System im Namen des Schülers) sendet einen formlosen Freitext-Wunsch (z. B. "Detaillierter Einstieg in Python Security und Penetration Testing für Netzwerke"). Das API-Gateway nimmt die Anfrage entgegen, erzeugt einen Eintrag in der Courses-Tabelle (mit status: generating) und startet asynchron den Haupt-Temporal-Workflow. Das API-Gateway antwortet dem Client sofort mit einem 202 Accepted.
2. **Struktur-Generierung (Activity 1 - Curriculum):** Der Workflow triggert eine Aktivität, die die OpenRouter API aufruft. Mittels der instructor-Bibliothek wird der Output des Frontier-LLMs in das strikte JSON-Schema gezwungen, welches die Module und Lektionen definiert. Die Aktivität speichert diese Hierarchie relational in die Datenbank und registriert im Workflow eine Kompensationsfunktion (z.B. deleteCourseAndModules(course_id)), falls spätere Schritte kritisch fehlschlagen.
3. **Inhalt & Skript-Generierung (Activity 2 - Content Expansion):** Für jede im JSON generierte Lektion erstellt das LLM nun die eigentlichen Inhalte: ein detailliertes Teleprompter-Skript für den Video-Avatar sowie ausführliche Begleittexte und Code-Beispiele. Die fertigen Texte werden in semantisch sinnvolle Chunks zerlegt, mittels eines Embedding-Modells (z.B. text-embedding-3-small) vektorisiert und über eine einzige Transaktion zusammen mit den Lesson-Metadaten in die Postgres Embeddings-Tabelle geschrieben. Auch hier wird eine deleteEmbeddings-Kompensation im Saga-Log registriert.
4. **Video Rendering (Activity 3 - Long-Running Webhook):** Der Workflow iteriert über alle Lektionen und initiiert asynchrone Webhook-Requests an die HeyGen- oder Synthesia-API. Der Temporal-Workflow versetzt sich für die jeweilige Lektion in einen ressourcenschonenden Wartezustand (Await-Pattern), bis der externe Webhook-Call via Signal an Temporal die Fertigstellung meldet (Event: avatar_video.success). Die Signaturen der Webhooks werden auf Edge-Ebene verifiziert, bevor das Signal an Temporal durchgestellt wird.
5. **Kompensation (Der Fehlerfall):** Schlägt die Videogenerierung nach multiplen

automatischen Retries endgültig fehl (z.B. wegen API-Ausfällen oder wiederholten Inhaltsverstößen bei der externen Moderations-Engine der Video-Provider), fängt die Saga den Fehler auf. Alle im Workflow zuvor hinterlegten Kompensations-Routinen werden nun rückwärts ausgeführt: Löschung der verwaisten Vektoren in pgvector, Löschung der Inhaltsfragmente und Versetzen des Kurses in den Status failed. Das System bleibt zu jeder Zeit konsistent.

6. **Dashboard-Freigabe (Activity 4 - Human in the Loop):** Sind alle Lektionen und Videos erfolgreich generiert und verarbeitet, setzt der Workflow den Kurs-Status auf `pending_approval`. Das minimalistische Admin-Dashboard signalisiert dem Betreiber den Abschluss der Generierung. Ein einziger Klick des Betreibers (1-Click-Approval) ändert den Status auf `active` und der personalisierte E-Learning-Kurs wird automatisch via Drip-Campaign in das Dashboard des Schülers ausgespielt.

Dieser strikt Code-First-basierte Ablaufplan stellt sicher, dass Entwickler, Tools und KI-Agenten die isolierten Aktivitäten iterativ und sicher verbessern können, ohne die übergeordnete Orchestrierungssicherheit oder Systemintegrität jemals zu gefährden. Das System überwacht sich selbst, heilt asynchrone Fehlerzustände eigenständig, eliminiert inkonsistente Datenbankfragmente und reduziert den betrieblichen Wartungsaufwand auf ein absolutes Minimum.

Referenzen

1. pgque/blueprints/SPECx.md at main - GitHub, <https://github.com/NikolayS/pgque/blob/main/blueprints/SPECx.md>
2. The Backend Shift: Leveraging Open Source Powerhouses for Faster, Leaner Apps, <https://dev.to/glennlaysonjr/the-backend-shift-leveraging-open-source-powerhouses-for-faster-leaner-apps-11ha>
3. How to Integrate AI Agents with Temporal Workflows | Fastio, <https://fast.io/resources/ai-agent-temporal-integration/>
4. Of course you can build dynamic AI agents with Temporal, <https://temporal.io/blog/of-course-you-can-build-dynamic-ai-agents-with-temporal>
5. How to build deep research agents using Temporal and Braintrust, <https://temporal.io/blog/how-to-build-deep-research-agents-using-temporal-and-braintrust>
6. 50+ PostgreSQL Jobs in Mumbai - Cutshort, <https://cutshort.io/jobs/postgresql-jobs-in-mumbai>
7. Temporal: Durable Execution Solutions, <https://temporal.io/>
8. How Rapidflare built a million+ document ingestion pipeline for Agents on Temporal, <https://temporal.io/blog/how-rapidflare-built-a-million-document-ingestion-pipeline-for-agents-on-temporal>
9. I built OpenWorkflow: a lightweight alternative to Temporal (Postgres/SQLite) - Reddit,

- https://www.reddit.com/r/javascript/comments/1r18ld3/i_built_openworkflow_a_lightweight_alternative_to/
10. Saga Pattern | Temporal Platform Documentation,
<https://docs.temporal.io/design-patterns/saga-pattern>
 11. Automating the Saga Pattern with Temporal,
<https://pages.temporal.io/rs/250-WIU-007/images/tech-guide-saga-pattern-made-easy.pdf>
 12. Saga Design Pattern Explained for Distributed Systems - Temporal,
<https://temporal.io/blog/saga-pattern-made-easy>
 13. Support for Saga compensating transactions in Typescript - Temporal Community Forum,
<https://community.temporal.io/t/support-for-saga-compensating-transactions-in-typescript/6392>
 14. Saga Compensating Transactions - Temporal,
<https://temporal.io/blog/compensating-actions-part-of-a-complete-breakfast-with-sagas>
 15. Observability & Tracing for Instructor - Litefuse,
<https://litefuse.ai/integrations/frameworks/instructor>
 16. Harnessing Structured Outputs: Elevating AI with OpenAI's Instructor Library | by NAITIVE,
<https://medium.com/@NAITIVE/harnessing-structured-outputs-elevating-ai-with-openais-instructor-library-7573fe992b01>
 17. Structured outputs with OpenRouter, a complete guide with instructor,
<https://python.useinstructor.com/integrations/openrouter/>
 18. Structured outputs with OpenAI, a complete guide with instructor,
<https://python.useinstructor.com/integrations/openai/>
 19. Ollama Managed Hosting mit GPU Servern in Deutschland - ennoia.ai,
<https://ennoia.ai/ollama-hosting/>
 20. Ollama selbst hosten 2026: Kosten, Hardware & DSGVO - webAION,
<https://webaion.de/ratgeber/self-hosted-ki-ollama-unternehmen>
 21. Ollama + Open WebUI 2026: Lokale LLMs für DSGVO-konforme KI - Qytera,
<https://www.qytera.de/blog/compliance-sichere-ai-fuer-unternehmen>
 22. Managed Ollama & vLLM KI Server aus Deutschland - WZ-IT,
<https://wz-it.com/ai-server/>
 23. Ollama und LM Studio - die DSGVO-konformen Alternativen zu ChatGPT,
<https://www.it-schulungen.com/seminare/kunstliche-intelligenz/ki-chatbot/ollama-und-lm-studio-die-dsgvo-konformen-alternativen-zu-chatgpt.html>
 24. vLLM Docker Deployment: Production-Ready Setup Guide (2026) | Inference.net,
<https://inference.net/content/vllm-docker-deployment/>
 25. vLLM Quickstart: High-Performance LLM Serving - in 2026 - Glukhov.org,
<https://www.glukhov.org/llm-hosting/vllm/vllm-quickstart/>
 26. Deploying vLLM with Docker: The Complete Guide to Production-Ready LLM Inference | by Juan C Olamendy | Medium,
<https://medium.com/@juanc.olamendy/deploying-vllm-with-docker-the-complete-guide-to-production-ready-llm-inference-39b812c01535>

27. vLLM Production Deployment 2026: Multi-GPU Tensor Parallel + FP8 Docker Setup on H100 | Spheron Blog,
<https://www.spheron.network/blog/vllm-production-deployment-2026/>
28. Ollama: Die Referenz-Architektur für souveräne, private Large Language Models (LLMs),
<https://ayedo.de/posts/ollama-die-referenz-architektur-fur-souverane-private-large-language-models-llms/>
29. Self-Hosted LLM on Kubernetes: A Production vLLM Deployment - René Zander,
<https://renezander.com/blog/self-hosted-llm-kubernetes/>
30. Synthesia API: What Developers Actually Need to Know Before Building With It,
<https://autogpt.net/synthesia-api/>
31. How to Use AI Avatars for Content Creation: HeyGen Voice Mirroring and Agent Features,
<https://www.mindstudio.ai/blog/ai-avatars-content-creation-heygen-voice-mirroring>
32. AI Video API Guide: Text-to-Video and Image-to-Video (2026) | PixVerse,
<https://pixverse.ai/en/blog/ai-video-api-guide-text-to-video-image-to-video-solutions>
33. 6 Most Powerful AI Video Generation APIs in 2026 - Creatify AI,
<https://creatify.ai/blog/most-powerful-ai-video-generation-apis>
34. Webhooks - HeyGen Documentation,
<https://developers.heygen.com/docs/webhooks>
35. Examples - HeyGen Documentation, <https://developers.heygen.com/examples>
36. Webhook Events - HeyGen Documentation,
<https://developers.heygen.com/docs/webhook-events>
37. Prompt to Video - HeyGen Documentation,
<https://developers.heygen.com/docs/video-agent>
38. Vector Database Comparison: Pinecone vs Qdrant vs Weaviate vs pgvector in Production, <https://tensoria.fr/en/blog/vector-database-comparison>
39. Best vector databases for RAG in 2026 - Articles - Braintrust,
<https://www.braintrust.dev/articles/best-vector-databases-for-rag-2026>
40. PostgreSQL 18 + pgvector: The Definitive Guide to Building Production-Grade RAG Pipelines | by Mohit soni | Medium,
<https://medium.com/@mohitsoni/postgresql-18-pgvector-the-definitive-guide-to-building-production-grade-rag-pipelines-239ee9c0e56f>
41. pgvector Guide: Vector Search and RAG in PostgreSQL - Encore Cloud,
<https://encore.dev/blog/you-probably-dont-need-a-vector-database>
42. Beyond Similarity Search: A Unified Data Layer for Production RAG Systems - arXiv, <https://arxiv.org/pdf/2605.03275>
43. Multitenancy - Qdrant,
<https://qdrant.tech/documentation/manage-data/multitenancy/>
44. Build a RAG System with pgvector on Managed PostgreSQL (2026) | DanubeData,
<https://danubedata.ro/blog/pgvector-rag-managed-postgres-2026>
45. SessionHeartBeatEnable (Function) - PC SOFT - Online documentation,
<https://doc.windev.com/en-US/?1000026248&name=SessionHeartBeatActive>

46. (PDF) Tamper-Proof Operational Activity Tracking System - ResearchGate, https://www.researchgate.net/publication/407275974_Tamper-Proof_Operational_Activity_Tracking_System
47. JWT Authentication: A Comprehensive Guide for Developers - Authgear, <https://www.authgear.com/post/jwt-authentication-a-secure-scalable-solution-for-modern-applications/>
48. Authenticate a session - Stytych Docs, <https://stytych.com/docs/multi-tenant-auth/manage-sessions/validate-session>
49. JWT Validation Best Practices for Secure Token Authentication - LoginRadius, <https://www.loginradius.com/blog/identity/jwt-validation-checklist-tokens>
50. Guide to Understanding JWT Validation (+Code Examples), <https://idura.eu/blog/jwt-validation-guide>
51. JWT Authentication: Building a Hybrid Model That Actually Works - ralf-lang.de, <https://www.ralf-lang.de/2026/03/09/jwt-authentication-building-a-hybrid-model-that-actually-works/>